LANCE : architecture du pipeline et méthodes de preuve

Codex, agent de diagnostic statique

Dernière révision : 2026-10-07 13:41 (Europe/Brussels, UTC+02:00)

Exploration et protocoles proposés ; aucune expérimentation nouvelle

Résumé

Comment enrichir l'approche de LANCE avec une contribution vérifiable ? Une lecture statique ciblée du code et des études existantes compare quatre architectures : phases ordonnées, dossiers explicites de preuves et d'hypothèses, comparaison différentielle de configurations, validation indépendante. Le code contient déjà un registre de candidats, une attribution des appels d'outils et une restitution déterministe. La difficulté est de garantir que chaque conclusion conserve ses observations, ses contradictions et ses limites : certains résultats d'outils sont filtrés avant leur journalisation. Les trois extensions proposées peuvent d'abord être étudiées sur dossiers figés, sans nouvelle collecte réseau. La copie de travail contient des conflits ; la lecture ne démontre ni son exécutabilité, ni un gain de détection ou de fiabilité.

Table des matières

1 Question et périmètre **1**

2 Méthode et état des sources **2**

3 Constats : une base déjà structurée **2**

4 Quatre architectures à comparer **3**

5 Hypothèses et protocole proposé **5**

6 Limites et conclusion **5**

Références **6**

1 Question et périmètre

Une approche intéressante doit permettre de tester une proposition scientifique, par exemple : une représentation explicite des preuves réduit-elle les conclusions injustifiées et facilite-t-elle la revue humaine ? Le nombre d'agents ou de phases est un choix d'architecture ; il ne constitue pas une mesure de qualité de l'audit.

Ce rapport examine la circulation des observations et la formation des conclusions dans LANCE, outil d'audit de réseaux de l'Internet des objets (IoT). Il traite l'attribution, l'incertitude, les interruptions et l'isolation. Il propose des études défensives sur données autorisées et déjà collectées ou synthétiques. Aucun changement applicatif, appel au modèle LANCE, audit réseau, exécution de laboratoire ou déploiement n'est réalisé.

Les recherches sur l'apport du modèle et la conservation du contexte ont déjà proposé de comparer règles et modèle sur observations communes, avec des cas sains et un coût suivi [1]. Le présent diagnostic précise les points d'appui actuels et distingue quatre variantes d'architecture. Il ne remplace pas leurs protocoles détaillés.

2 Méthode et état des sources

Lecture statique. Les index research/README.md et docs/README.md, les guides de phases et les études liées sont consultés, puis les propriétaires du routage, de l'agrégation, du journal et du rapport. Les constats ci-dessous décrivent les instructions visibles dans ces fichiers. Les chemins et lignes permettent de revenir à chaque observation ; les numéros peuvent changer lors d'une modification ultérieure [2].

Version étudiée. La base Git est a8dbeb9404f0cad960d61e42f02a1d70ca143a59. La copie de travail contient des modifications locales et des conflits non résolus, notamment dans src/agent/phases/verification/run.py, src/agent/exploit_evidence.py et plusieurs index. Le commit seul ne reproduit donc pas tous les textes lus. Les branches en conflit ne sont ni choisies, ni fusionnées. Aucun module applicatif n'est importé, aucun test applicatif exécuté et aucun comportement du système déployé inféré de cette lecture.

Sources externes. Le résumé officiel du guide NIST SP 800-115 présente la planification des examens, l'analyse des constats et la préparation de mesures correctives ; seul ce résumé et sa notice sont lus ici [4]. La note W3C *PROV-Overview* décrit l'origine des données, les acteurs et les transformations ; l'introduction et le guide de lecture sont consultés [5]. Ce sont des appuis de vocabulaire. Ils ne démontrent pas que les architectures proposées améliorent LANCE. La revue est ciblée, sans prétention d'exhaustivité bibliographique.

3 Constats : une base déjà structurée

3.1 Phases ordonnées et états distincts

Le registre route six phases : graphe, reconnaissance, analyse, vérification, évaluation des accès, rapport (src/agent/phases/registry.py, l. 11–26). Le moteur les trie puis les parcourt ; il contrôle arrêt, prérequis et conditions d'exécution (src/agent/pipeline.py, l. 663–753). L'ordre des phases est donc visible dans le code. Cela n'implique pas que tous les travaux internes soient séquentiels : le scanner prévoit plusieurs tâches de travail selon le contexte.

Les états terminaux distinguent notamment arrêt, blocage, résultat partiel et échec (src/agent/results.py, l. 4–19). Un état inconnu ne compte pas comme succès. Ces états décrivent l'exécution ; la fin d'une phase ne prouve aucune propriété de sécurité. De même, l'admission structurelle d'un candidat ne certifie ni sa preuve ni sa compatibilité avec une vulnérabilité connue (src/agent/finding_schema.py, l. 1–45).

3.2 Provenance et conservation des informations

Le registre de candidats conserve la valeur initiale, le fichier et le type de source, l'indice, la décision et son motif. Il crée une copie de travail pour la normalisation et garde les candidats mal formés avec leur rejet explicite (src/agent/phases/analysis/aggregation_loading.py, l. 20–51). Une projection brute distincte est annoncée dans l'agrégation (src/agent/phases/analysis/aggregation.py, l. 81–109). C'est un point d'appui pour expliquer pourquoi une proposition a été retenue ou écartée.

Le scanner marque ses appels decision_source="rules" (src/agent/phases/analysis/scan.py, l. 35–55). Le journal commun enregistre horodatage, durée, arguments, résultat, phase, agent, référence de preuve et source de décision ; un échec d'écriture est explicite (src/agent/core/executor.py, l. 74–100).

Ce mécanisme permet de distinguer une observation produite par une règle d'une proposition du modèle. Il ne mesure pas, à lui seul, l'effet causal de chaque acteur sur le résultat final.

Limite vérifiée. Le chargeur d'outils peut filtrer des lignes et tronquer la sortie avant de la retourner (src/agent/tools/tool_loader.py, l. 228–243). L'exécuteur archive cette valeur retournée (src/agent/core/executor.py, l. 126–133). Sur ce chemin, le journal conserve donc une sortie déjà transformée, pas nécessairement la capture native intégrale. Une référence stable ne permet pas de retrouver une portion jamais conservée. L'effet réel sur les conclusions reste à mesurer.

3.3 Frontières de restitution, d'arrêt et d'accès

L'index du rapport distingue référence absente, ambiguë, incohérente avec l'identité ou la destination. Il indique explicitement qu'il ne valide pas la propriété de sécurité (src/agent/phases/report/traceability.py, l. 1–69). Le rapport travaille par fiches, calcule sa synthèse et assemble les faits déterministement. La rédaction d'une fiche est un appel sans outil ; sa cause d'indisponibilité et les erreurs d'assemblage restent visibles (src/agent/phases/report/run.py, l. 57–118, 169–201). Cette séparation évite d'attribuer à une rédaction réussie la valeur d'une nouvelle preuve.

Le contrôle d'exécution vérifie arrêt et budgets avant les outils, puis refuse de poursuivre un appel lorsqu'un arrêt est déjà visible (src/agent/core/executor.py, l. 30–49, 103–125). Ces contrôles ne démontrent pas l'annulation immédiate d'un processus déjà en cours. Les chemins de livrables sont limités au dossier du run et plusieurs fichiers internes sont masqués aux outils de l'agent (src/agent/artifacts.py, l. 6–67). La documentation précise que ces frontières ne sont pas un bac à sable système et que certains états, notamment le graphe, restent partagés [3].

Table 1 – Acquis visibles et extensions proposées. Les acquis sont des constats de lecture, pas des garanties vérifiées en exécution.

Sujet	Visible dans les sources lues	Extension à étudier
Candidats	Valeur initiale, origine, décisions d'agrégation.	Relier également les alternatives, contradictions et inconnues.
Preuves	Références et sorties retournées dans un journal.	Conserver la capture avant projection, ou déclarer précisément son absence.
Attribution	Source de décision et agent enregistrés.	Comparer des interprètes sur les mêmes dossiers ; distinguer collecte et interprétation.
Restitution	Faits assemblés par le code, fiches sans outils.	Rendre chaque conclusion contestable par un dossier de preuves lisible.
Exécution	États, limites, dossiers de livrables et refus explicites.	Vérifier l'arrêt en cours d'appel et l'isolation de tous les états par tests dédiés.

4 Quatre architectures à comparer

4.1 Phases ordonnées : référence de comparaison

La première variante conserve le chemin actuel et les contrats de preuve. Son intérêt est de fournir une référence compréhensible : observations, candidats, vérification, restitution. Sa limite est qu'un livrable compact peut omettre une alternative utile à une étape ultérieure. Le diagnostic ne démontre pas que cette perte se produit sur tous les cas. Avant comparaison, il faut figer une version exécutable résolue et documenter quelles informations chaque phase reçoit effectivement. Ajouter une phase sans préciser ce qu'elle mesure ne constituerait pas une nouvelle contribution scientifique.

4.2 Dossiers de preuves et d'hypothèses explicites

Proposition. Chaque propriété examinée reçoit un dossier : énoncé précis, objet concerné, observations pertinentes, origine et date, éléments en faveur et en contradiction, inconnues, règle de conclusion et historique des révisions. Une observation décrit ce qui a été recueilli ; une hypothèse décrit ce qu'on en déduit. La conclusion peut rester indéterminée. Le modèle explique ces dossiers, tandis que les règles retenues restent responsables de l'acceptation des preuves.

Cette variante prolonge le registre existant. Le vocabulaire de provenance de W3C PROV suggère de représenter séparément données, transformations et acteurs [5]. Cette adaptation à LANCE est une proposition de ce rapport, sans revendication de conformité PROV. Il faut notamment conserver les observations avant projection, indiquer leur caractère complet ou partiel et garder une vue compacte qui renvoie à ce qui existe réellement.

La figure 1 explicite une distinction centrale : une validation peut contester une conclusion sans modifier ses observations.

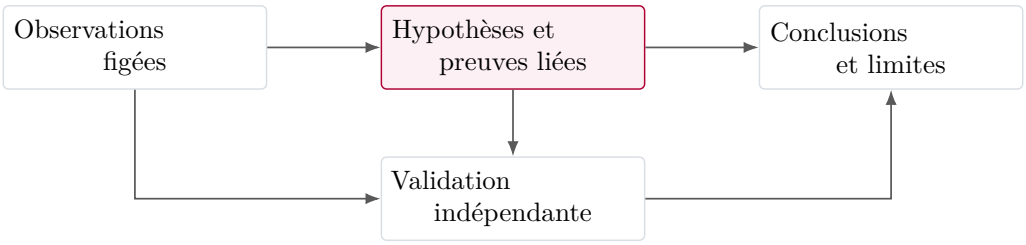


Figure 1 – Architecture proposée de restitution sur données figées. Le validateur lit les observations et les hypothèses, puis fournit une appréciation distincte ; il ne réécrit pas les observations. Aucune collecte réseau n'est représentée.

4.3 Audit différentiel : configurations fragiles et corrigées

Proposition. Comparer deux dossiers issus de configurations contrôlées dont une propriété défensive varie : par exemple un inventaire ou une configuration de politique d'accès avant et après correction. Dans la première étude, utiliser des fichiers de configuration fournis et des observations synthétiques clairement déclarées ; aucune reproduction de faille n'est nécessaire. Les dossiers sont codés sans indiquer au classificateur lequel est le cas attendu sain. L'annotation de référence est tenue à part.

La question porte sur la cohérence : la conclusion relative à la propriété modifiée change-t-elle, tandis que les conclusions relatives aux propriétés inchangées restent stables ? Un résultat différent ne démontre une relation causale que si les autres changements pertinents sont maîtrisés. Un cas dit « corrigé » peut présenter d'autres risques ; une différence de bannière ou de nom ne doit pas suffire à changer le verdict. Cette variante mesure une sensibilité à une correction connue et les fausses alertes, au-delà du nombre brut de constats produits.

4.4 Validation indépendante des conclusions

Proposition. Une seconde lecture reçoit les observations originales et les règles d'acceptation gelées. Elle recherche les conclusions sans preuve, les mauvaises attributions et les contradictions non résolues. Son appréciation est publiée séparément ; un désaccord ouvre un point à revoir, sans supprimer silencieusement le candidat ni inventer une confirmation. Une revue humaine peut fournir une annotation indépendante pour le corpus étudié.

L'indépendance exige de préciser ce qui est partagé : observations, modules d'interprétation, taxonomie, règles et éventuellement famille de modèle. Deux agents utilisant le même parseur et la même règle erronée peuvent s'accorder. Un vote entre modèles ne remplace donc pas une référence

défensive documentée. La valeur recherchée est la détection d'erreurs de conclusion ; l'accord seul n'est pas la métrique de réussite.

Table 2 – Ce que chaque comparaison pourrait établir, sous conditions. Aucun gain n'est observé dans cette étude.

Variante	Question mesurable	Condition principale
Phases ordonnées	Quel est le comportement de référence ?	Version résolue, entrées et sorties effectivement figées.
Dossiers explicites	Les conclusions restent-elles reliées à leurs preuves ?	Mêmes observations et règles ; seule la représentation varie.
Différentielle	La réponse suit-elle une correction défensive connue ?	Variation contrôlée, annotation séparée et cas non concernés.
Validation indépendante	Des erreurs de conclusion sont-elles détectées ?	Référence de contrôle indépendante et dépendances partagées explicites.

5 Hypothèses et protocole proposé

L'étude minimale peut être réalisée hors ligne. Elle porte sur la qualité des conclusions et de leur justification ; elle ne mesure pas l'exploration réseau. Trois hypothèses sont réfutables : les dossiers explicites réduisent les affirmations sans preuve ; les paires différentielles révèlent des changements de conclusion incohérents ; la validation séparée repère plus d'erreurs que la seule vérification de références. Aucun de ces effets n'est acquis.

Le protocole proposé comporte quatre étapes :

1. **Fixer les observations.** Préparer un corpus de dossiers autorisés et existants, ou de dossiers synthétiques déclarés, incluant cas corrects, preuves manquantes, contradictions et mauvaises attributions. Consigner la provenance, les portions absentes et la version de chaque fichier.
2. **Fixer la référence.** Faire annoter propriétés et exigences de preuve avant comparaison ; résoudre les désaccords humains et conserver les cas indéterminés. Une simple sortie attendue du code existant ne fournit pas une validation indépendante de sa sémantique.
3. **Varier une dimension.** Comparer représentation actuelle et dossiers explicites avec le même interprète. Ajouter ensuite la lecture indépendante, puis les paires défensives. Inclure un interprète à règles et, si le coût et l'accès le permettent, un modèle de langage sans outil. Garder constantes les autres entrées et les exigences de preuve.
4. **Publier erreurs et coûts.** Compter les conclusions injustifiées, les liens de preuve exacts, les contradictions oubliées, les fausses alertes et les abstentions. Mesurer aussi temps de revue, coût modèle et issues incomplètes. Publier les dénominateurs et valeurs manquantes ; les instances partageant un même mécanisme ne sont pas des observations indépendantes.

Une amélioration de liens de preuve peut accompagner une hausse des abstentions. Il faut donc définir avant comparaison ce qui constitue une amélioration utile, et publier cette contrepartie. Des données synthétiques réussies justifieraient au plus la poursuite sur des dossiers réels autorisés ; elles ne démontreraient pas la qualité d'un audit en production.

6 Limites et conclusion

La lecture ne fournit aucun taux d'erreur, coût expérimental ou temps humain mesuré. Les constats de code sont limités aux chemins inspectés ; les modules en conflit empêchent de présenter cette copie comme une référence exécutable. Les études antérieures décrivent leurs propres dates et expériences ; leurs résultats ne sont pas rejoués ici. Les architectures proposées n'ont été ni implémentées, ni comparées.

Une garantie complète d'isolation demanderait de vérifier état du graphe, processus, fichiers, entrées publiques et journaux. Une interruption fiable demanderait de distinguer refus d'un prochain appel, arrêt d'un appel en cours et finalisation avec les données déjà disponibles. Ces objectifs restent des conditions de validation logicielle ; une réussite aux tests ne prouverait pas une amélioration scientifique de l'audit.

Conclusion de conception. LANCE dispose déjà de frontières pertinentes entre candidats, observations, exécution et restitution. La piste la plus directement étayée par ce diagnostic est de rendre les dossiers de preuve explicites et récupérables, puis d'évaluer la fidélité des conclusions sur observations communes. La comparaison différentielle et la validation indépendante apportent deux contrôles complémentaires. Leur intérêt reste une hypothèse mesurable. Le prochain travail utile est de figer les règles, le corpus et les critères de jugement avant toute affirmation de gain.

Références

[1] Recherches LANCE, *Agent / automatisation* et *Conservation du contexte*, rapports révisés le 1er octobre 2026 à 10 :23, Europe/Brussels. Sources et analyses ciblées consultées le 7 octobre 2026 ; expériences antérieures non rejouées. Dossier agent / automatisation ; dossier contexte.

[2] Dépôt LANCE, copie de travail inspectée le 7 octobre 2026, base a8dbeb9404f0cad960d61e42f02a1d70ca143a59, modifications locales et conflits non résolus. Les liens de fichiers et lignes dans les constats sont la référence de lecture ; aucun résultat d'exécution nouvelle. Guide des phases, index documentaire et conventions de recherche.

[3] Documentation LANCE, *Livrables : un dossier explicite par run* et *Rapport rédigé par sections*. Pages de la copie de travail consultées le 7 octobre 2026, pas une attestation de validation actuelle ni de déploiement.

[4] K. Scarfone, M. Souppaya, A. Cody et A. Orebaugh, *Technical Guide to Information Security Testing and Assessment*, NIST SP 800-115, septembre 2008. https://csrc.nist.gov/pubs/sp/800/115/final. Notice et résumé officiel lus le 7 octobre 2026 ; guide complet non dépouillé.

[5] P. Groth et L. Moreau (éd.), *PROV-Overview : An Overview of the PROV Family of Documents*, W3C Working Group Note, 30 avril 2013. https://www.w3.org/TR/2013/NOTE-prov-overview-20130430/. Introduction et guide de lecture consultés le 7 octobre 2026 ; aucune implémentation ni validation de conformité PROV réalisée.